claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-aa8f98c90fe46ebbc.jsonl`
- SHA-256: `5fbdcf6770203cf708803e7097cf7aac1f34596a9b62b7ffa1115ded7da22c2d`
- Source modified: `2026-06-14T09:04:32+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T08:59:16.953Z)

Implement a focused, mergeable engineering PR in the badminton-highlight-indexer repo (cwd C:/Users/avidu/Projects/badminton-highlight-indexer): a HELD-OUT-USER LEARNING-CURVE eval harness ? operationalizing "<K videos to superior quality" as a falsifiable release criterion for the personalization feature. Pure-CPU eval; reuse the shipped harness; do NOT reinvent stats.

FIRST read to use the real APIs correctly: backend/eval/experiment.py (VideoCandidates, Variant, compare, evaluate_variant, predict, and the leave-one-video-out semantics) and backend/eval/eval_stats.py (paired_delta_ci, paired_sign_test). Mirror the synthetic-VideoCandidates test pattern in tests/test_fusion_compare.py.

CONSTRUCT ? add backend/eval/per_user_eval.py with `learning_curve(videos, control_variant, candidate_variant, *, iou=0.5, min_k=2, seed=0)`: for a single user's list of VideoCandidates, sweep k = min_k .. len(videos)-1; at each k, compare control vs candidate on a held-out split (fit/score on k videos, evaluate on the remaining held-out videos) using the existing harness primitives + eval_stats.paired_delta_ci for the honest ?F1 CI + sign test. Return a structured result: per-k records {k, control_f1, candidate_f1, mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test} PLUS the headline `smallest_k_with_lift` = the smallest k where candidate beats control with CI clearing 0 AND the sign test agreeing (else None). Deterministic given seed; no GPU/torch/cv2; no file I/O in the function (callers pass VideoCandidates).

TESTS (tests/test_per_user_eval.py, GPU-free, synthetic VideoCandidates a la test_fusion_compare.py): (1) a corpus where the candidate clearly wins above some k -> smallest_k_with_lift is found + sensible; (2) a corpus where it never wins -> None; (3) per-k records carry the expected keys + honest CI/sign-test fields; (4) edge: too-few videos handled gracefully.

VERIFY: `python -m ruff check backend/eval/per_user_eval.py tests/test_per_user_eval.py`; `python -m mypy backend/eval/per_user_eval.py`; `python -m pytest tests/test_per_user_eval.py tests/test_fusion_compare.py tests/test_eval_stats.py -q`. All must pass.

Branch-safe ISOLATED worktree off origin/master: `git -C C:/Users/avidu/Projects/badminton-highlight-indexer fetch origin` then `git -C ... worktree add C:/Users/avidu/Projects/bhi-pueval -b feat/per-user-eval origin/master`. Commit explicit paths (end msg with `Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>`); push; `gh pr create --base master`; DO NOT merge. Return: PR number/URL, files changed, exact ruff/mypy/pytest results, worktree path.

2. assistant (2026-06-14T08:59:19.252Z)

I'll start by reading the key files to understand the real APIs, then implement the PR. Let me begin with the repo-freshness pull and reading the core eval files.

3. user (2026-06-14T08:59:22.211Z)

(Bash completed with no output)

4. user (2026-06-14T08:59:22.230Z)

```
1       """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3       The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4       non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5       report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6       """
7       import json
8       import os
9       from types import SimpleNamespace
10
11      import pytest
12
13      from backend.eval import fusion_compare
14      from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17      def _write_video(d, name):
18          """A video whose shuttle features are identical for pos/neg (no signal) but whose
19          players_in_court separates rallies (4) from junk (0)."""
20          cands = []
21          for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
22              shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES}                 # constant
-> non-separating
23              person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24              person["players_in_court"] = 4.0 if pos else 0.0                   # the
separating signal
25              person["in_court_ratio"] = 1.0 if pos else 0.0
26              cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27          data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                  "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29          with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30              json.dump(data, f)
31
32
33      def test_fusion_compare_reports_person_driven_lift(tmp_path):
34          d = str(tmp_path)
35          for k in range(6):
36              _write_video(d, f"vid{k}")
37          videos = fusion_compare.load_videos(d)
38          assert len(videos) == 6
39          res = fusion_compare.compare(videos, iou=0.5)
40          assert res["num_videos"] == 6
41          assert res["variant_b"]["honest_lovo"] is True                 # LOVO-honest learned
variant
42          # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43          assert res["delta"]["f1"] > 0.0
44          assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45          # format_result must not crash and must mention both variants.
46          out = fusion_compare.format_result(res, 0.5)
47          assert "shuttle-only" in out and "+person" in out
48
49
50      def test_fusion_compare_delta_stats_and_ci(tmp_path):
51          """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""
52          d = str(tmp_path)
53          for k in range(6):
```

```
            _write_video(d, f"vid{k}")
        res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
        ds = fusion_compare.delta_stats(res)
        assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
        assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9   # CI
brackets mean (±fp)
        assert ds["sign_test"]["wins"] == 6          # fusion wins on every video (person
feature separates)
        assert ds["ci_excludes_zero"] is True
        out = fusion_compare.format_result(res, 0.5)
        assert "CI" in out and "sign test" in out


    def test_fusion_compare_needs_two_videos(tmp_path):
        _write_video(str(tmp_path), "only")
        videos = fusion_compare.load_videos(str(tmp_path))
        assert len(videos) == 1  # compare() / experiment.compare needs >=2 for honest LOVO


    # ---------------------------------------------------------------- extractor assembly

    def test_extract_video_assembles_shuttle_and_person(monkeypatch):
        from backend.eval import fusion_golden
        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

        intervals = [(0.0, 10.0), (40.0, 41.0)]
        x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
        points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))
                  for i in range(10)]

        monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
                            lambda traj, fps, fw: (intervals, x_shuttle))
        monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
        monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])

        class _StubDetector:
            def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
                return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
                         "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
                        for (sf, ef) in windows]

        spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
                "labels": "foo.rallies.csv"}
        data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)

        assert data["name"] == "t"
        assert data["gts"] == [[0.0, 10.0]]
        assert len(data["candidates"]) == 2
        c0 = data["candidates"][0]
        assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
        assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
        assert c0["label"] == 1                     # (0,10) matches the gt -> positive
        assert data["candidates"][1]["label"] == 0  # (40,41) -> negative
        assert all(isinstance(v, float) for v in c0["person"].values())


    def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
        """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""
        from backend.eval import fusion_golden
        out_dir = str(tmp_path / "out")
```

```
        os.makedirs(out_dir)
        manifest = [
            {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
            {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
        ]
        manifest_path = str(tmp_path / "m.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(manifest, f)
        # a prior run already produced alpha's JSON
        with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
encoding="utf-8") as f:
            json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)

        extracted = []

        def _stub(spec, **kw):
            extracted.append(spec["name"])
            return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}

        monkeypatch.setattr(fusion_golden, "extract_video", _stub)

        fusion_golden.run(manifest_path, out_dir, fresh=False)
        assert extracted == ["beta"]                # alpha skipped (already extracted)

        extracted.clear()
        fusion_golden.run(manifest_path, out_dir, fresh=True)
        assert extracted == ["alpha", "beta"]       # --fresh redoes both


    def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
        """--smallest-first orders by trajectory size; default keeps manifest order."""
        from backend.eval import fusion_golden
        out_dir = str(tmp_path / "out")
        os.makedirs(out_dir)
        (tmp_path / "big.csv").write_text("x" * 2000)
        (tmp_path / "small.csv").write_text("x" * 20)
        manifest = [
            {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
            {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
        ]
        mp = str(tmp_path / "m.json")
        with open(mp, "w", encoding="utf-8") as f:
            json.dump(manifest, f)

        order = []

        def _stub(spec, **kw):
            order.append(spec["name"])
            return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}

        monkeypatch.setattr(fusion_golden, "extract_video", _stub)

        fusion_golden.run(mp, out_dir, smallest_first=True)
        assert order == ["small", "big"]            # smaller trajectory first

        order.clear()
        fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
        assert order == ["big", "small"]            # manifest order preserved
```

```
169
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
173         assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
and Avi.MP4"
174         assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
"a.mp4"
175         with pytest.raises(ValueError):
176             _derive_video_path({"name": "n", "labels": "weird.csv"})
177
```

5. user (2026-06-14T08:59:22.230Z)

```
1       """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3       The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4       single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5       and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6       ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7       turns those into an honest summary:
8
9       - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10        interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11        many *videos* it wins on (the distribution-free test the repo already leans on:
12        "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14        producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15        match, never a held-out window from the same video).
16
17      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20      """
21      from __future__ import annotations
22
23      import math
24      import random
25      import statistics
26      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28      Interval = Tuple[float, float]
29      FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32      def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                       n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34          """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36          ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37          resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38          """
39          vals = [float(v) for v in values]
40          n = len(vals)
41          if n == 0:
42              return (0.0, 0.0)
43          if n == 1:
44              return (vals[0], vals[0])
```

```
        rng = random.Random(seed)
        means: List[float] = []
        for _ in range(max(1, n_boot)):
            means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
        means.sort()
        alpha = (1.0 - confidence) / 2.0
        last = len(means) - 1
        lo = means[max(0, int(math.floor(alpha * last)))]
        hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
        return (lo, hi)


    def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
        """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
(C1)."""
        vals = [float(v) for v in values]
        n = len(vals)
        mean = statistics.mean(vals) if vals else 0.0
        std = statistics.pstdev(vals) if n > 1 else 0.0
        lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
        return {"mean": mean, "std": std, "n": float(n),
                "ci_low": lo, "ci_high": hi, "confidence": confidence}


    def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
        """Two-sided exact sign test over per-fold deltas (B ? A).

        ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
        excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
        distribution-free "did B win on enough *videos*?" check.
        """
        wins = sum(1 for d in deltas if d > eps)
        losses = sum(1 for d in deltas if d < -eps)
        ties = len(deltas) - wins - losses
        n_eff = wins + losses
        if n_eff == 0:
            p = 1.0
        else:
            k = min(wins, losses)
            tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
            p = min(1.0, 2.0 * tail)
        return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
                "n_effective": float(n_eff), "p_value": p}


    def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
                        eps: float = 0.0) -> Dict[str, Any]:
        """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).

        ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
        fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
        test agrees ? not when a single mean ticked up.
        """
        n = min(len(a_values), len(b_values))
        deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
        summary = mean_with_ci(deltas, confidence, n_boot, seed)
        return {
            "mean_delta": summary["mean"],
            "ci_low": summary["ci_low"],
            "ci_high": summary["ci_high"],
            "n": float(n),
```

```
107             "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
108             "sign_test": paired_sign_test(deltas, eps),
109         }
110 
111 
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114 
115         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116         first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117         """
118         seen: List[Any] = []
119         for g in groups:
120             if g not in seen:
121                 seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130 
131 
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134 
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
137         scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150 
```

6. user (2026-06-14T08:59:22.601Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2 
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8 
9         - `backend.eval.metrics`       ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
```

```
  - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
  - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
  - `backend.eval.calibration`    ? `pct_improvement`

The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
(per-frame signals ? run once on the GPU/WSL box, persisted into
`candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
given persisted candidate features it produces `List[Interval]` deterministically,
with no GPU and zero production risk. So most experiments are pure offline
re-scoring of stored data and never touch the served pipeline.

Three modes (`EXPERIMENT_HARNESS.md` §5):
  - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
    aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
    held-out loop, but variant-parameterised.
  - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
    variants agree / diverge (A-only, B-only, agreed). The divergences are what a
    human spot-checks (and what two highlight reels would show side by side).
  - the promotion gate stays `run_regression.py` + `regression_baseline.json`
    (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

Pure + offline + unit-tested. The only DB-touching helper is
`load_video_candidates`, which reads persisted candidates via
`Database.query_candidate_segments`.
"""
from __future__ import annotations

import json
import logging
import os
from dataclasses import dataclass, field
from datetime import datetime, timezone
from hashlib import sha1
from typing import Any, Callable, Dict, List, Optional, Sequence

from backend.eval.calibration import pct_improvement
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import Interval, match_intervals, score
from backend.eval.regression import gt_hash
from backend.eval.training import label_candidates
from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

logger = logging.getLogger(__name__)

# Where a comparison run appends one reproducible record. Tracked location (NOT under
# the gitignored output/) so the leaderboard survives a clean checkout, mirroring
# regression.DEFAULT_BASELINE_PATH.
DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
_METRICS = ("precision", "recall", "f1", "mean_iou")


class ExperimentError(ValueError):
    """A variant cannot be evaluated as configured (e.g. a learned variant asked to
    score an unlabeled video with no pinned model, or a gate referencing an absent
    feature)."""


# ---------------------------------------------------------------------- Variant

```

```
75
76      @dataclass
77      class Variant:
78          """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80          A variant turns one video's persisted candidate windows + features into rally
81          intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82          filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83          to control ? the core no-regression guarantee.
84
85          Fields:
86            - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87              for the leaderboard and for selecting the served path (the existing
88              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89              New cues are recorded here default-OFF.
90            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96              cue persists that feature).
97            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99              required for `compare_unlabeled` on a learned variant.
100           - ``feature_names`` ? the persisted features the learned filter consumes. When set
101             WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102             (honest) ? the recipe, not a pinned artifact.
103           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105         """
106
107         name: str
108         segmenter: str = "wasb_hybrid"
109         config_overrides: Dict[str, Any] = field(default_factory=dict)
110         cue_flags: Dict[str, bool] = field(default_factory=dict)
111         min_crossings: int = 0
112         min_players: int = 0
113         classifier_model: Optional[str] = None
114         feature_names: Optional[List[str]] = None
115         threshold: float = 0.5
116         merge_gap: float = 1.0
117         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
125             return self.classifier_model is not None or bool(self.feature_names)
126
127         def to_dict(self) -> dict:
128             return {
129                 "name": self.name,
130                 "segmenter": self.segmenter,
131                 "config_overrides": self.config_overrides,
132                 "cue_flags": self.cue_flags,
133                 "min_crossings": self.min_crossings,
134                 "min_players": self.min_players,
```

```
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
```

```


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
```

```
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use
```

```
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
334                     assert model is not None                       # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo:                          # honest=False ? one model over all
videos
337                 model = all_model
338             else:                                      # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None                  # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                      for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
```

```
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }


    def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
        """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
        golden corpus and reports the **B ? A** deltas, per video and in aggregate.

        The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
        learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
        `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

        ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
        paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
        existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
        shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
        bare LOVO mean at n?6 is not trustworthy on its own.
        """
        if len(videos) < 1:
            raise ExperimentError("compare needs at least one labeled video")
```

```
        eval_a = evaluate_variant(videos, variant_a, iou, honest)
        eval_b = evaluate_variant(videos, variant_b, iou, honest)

        delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
        delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}

        by_video_a = {p["video"]: p for p in eval_a["per_video"]}
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]

        # One fingerprint over the whole label set ? detects "the deltas were measured
against
        # different labels" the same way regression.gt_hash guards the no-regression
baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ------------------------------------------------------------------ SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

```

```
        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
    the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
```

```
545                               feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

7. assistant (2026-06-14T08:59:25.653Z)

Now let me look at the supporting modules (`metrics`, `training`) and existing tests for eval_stats to use the real APIs correctly.

8. user (2026-06-14T08:59:26.598Z)

```
1       """Tests for backend.eval.eval_stats (course-corrections C1 + C3).
2
3       Pure/offline + deterministic (the bootstrap takes a fixed seed). Pins: CIs bracket the
4       mean and degrade sanely at tiny n; the sign test's exact p-value; leave-one-group-out
5       folds; and that out-of-fold predictions are genuinely out of fold (no leakage).
6       """
7       import pytest
8
9       from backend.eval.eval_stats import (
10          bootstrap_ci,
11          grouped_folds,
12          mean_with_ci,
13          out_of_fold_predictions,
14          paired_delta_ci,
15          paired_sign_test,
16      )
17
18
19      def test_bootstrap_ci_is_deterministic_and_brackets_mean():
20          vals = [0.5, 0.6, 0.7, 0.55, 0.65, 0.6]
21          lo1, hi1 = bootstrap_ci(vals, seed=0)
22          lo2, hi2 = bootstrap_ci(vals, seed=0)
23          assert (lo1, hi1) == (lo2, hi2)                      # same seed ? identical
24          assert lo1 <= sum(vals) / len(vals) <= hi1
25
26
27      def test_bootstrap_ci_degenerate():
28          assert bootstrap_ci([]) == (0.0, 0.0)
29          assert bootstrap_ci([0.42]) == (0.42, 0.42)
30          assert bootstrap_ci([0.5, 0.5, 0.5]) == (0.5, 0.5)  # no spread ? point interval
31
```

```

    def test_mean_with_ci_shape():
        out = mean_with_ci([0.4, 0.6, 0.5], seed=1)
        assert out["mean"] == pytest.approx(0.5)
        assert out["n"] == 3.0
        assert out["ci_low"] <= out["mean"] <= out["ci_high"]


    def test_sign_test_all_wins_exact_p():
        res = paired_sign_test([0.1, 0.2, 0.05, 0.3, 0.15, 0.2])   # 6 wins
        assert res["wins"] == 6.0 and res["losses"] == 0.0
        assert res["p_value"] == pytest.approx(2 * 0.5 ** 6)       # 0.03125


    def test_sign_test_ties_and_mixed():
        res = paired_sign_test([0.0, 0.0, 0.0])
        assert res["ties"] == 3.0 and res["n_effective"] == 0.0 and res["p_value"] == 1.0
        mixed = paired_sign_test([0.1, -0.1, 0.2])
        assert mixed["wins"] == 2.0 and mixed["losses"] == 1.0


    def test_paired_delta_ci_detects_consistent_improvement():
        a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
        b = [0.60, 0.66, 0.69, 0.61, 0.70, 0.63]            # B beats A every fold
        out = paired_delta_ci(a, b, seed=0)
        assert out["mean_delta"] > 0.0
        assert out["sign_test"]["wins"] == 6.0
        assert out["ci_excludes_zero"] is True


    def test_grouped_folds_leave_one_group_out():
        groups = ["m1", "m1", "m2", "m2", "m3"]
        folds = grouped_folds(groups)
        assert len(folds) == 3
        train0, test0 = folds[0]                             # first-seen group m1 held out
        assert test0 == [0, 1] and train0 == [2, 3, 4]
        assert grouped_folds(["only"]) == []                # single group ? no fold


    def test_out_of_fold_predictions_are_out_of_fold():
        groups = ["a", "a", "b", "b"]
        y = [0, 0, 1, 1]

        def fit_predict(train_idx, test_idx):
            # "Model" = mean label of the training groups; returned for each test item.
            m = sum(y[i] for i in train_idx) / len(train_idx)
            return [m] * len(test_idx)

        oof = out_of_fold_predictions(groups, fit_predict)
        # Group a's OOF uses only group b (mean 1.0) and vice-versa ? never its own labels.
        assert oof == [1.0, 1.0, 0.0, 0.0]

```

9. user (2026-06-14T08:59:26.777Z)

```
    """Pure temporal interval-matching metrics for rally evaluation.

    No I/O, no DB, no global state ? every function here is deterministic and unit
    tested. An "interval" is a `(start, end)` tuple of floats in seconds with
    start < end. We score a list of *predicted* rally intervals against a list of
    *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
    """

    from dataclasses import dataclass, field
    from typing import List, Tuple, Optional
```

```
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
24              return 0.0
25          inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26          if inter <= 0.0:
27              return 0.0
28          union = (a_end - a_start) + (b_end - b_start) - inter
29          return inter / union if union > 0 else 0.0
30
31
32      @dataclass
33      class Match:
34          """One matched predicted?ground-truth pair."""
35          pred_index: int
36          gt_index: int
37          iou: float
38
39
40      @dataclass
41      class MatchResult:
42          """Outcome of matching predictions to ground truth at a given IoU threshold."""
43          matches: List[Match] = field(default_factory=list)
44          unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
45          unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)
46
47
48      def match_intervals(preds: List[Interval], gts: List[Interval],
49                          iou_threshold: float = 0.5) -> MatchResult:
50          """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52          Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53          are claimed highest-IoU first, and each prediction/GT can be used once. This
54          keeps a single prediction from being credited for two ground-truth rallies
55          (and vice versa).
56          """
57          candidates = []
58          for pi, p in enumerate(preds):
59              for gi, g in enumerate(gts):
60                  iou = interval_iou(p, g)
61                  # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62                  # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63                  if iou > 0.0 and iou >= iou_threshold:
64                      candidates.append((iou, pi, gi))
65          # Highest IoU first; ties broken by indices for determinism.
66          candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68          used_pred, used_gt = set(), set()
69          matches: List[Match] = []
70          for iou, pi, gi in candidates:
71              if pi in used_pred or gi in used_gt:
72                  continue
73              used_pred.add(pi)
```

```
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
        false_negatives: int
        precision: float
        recall: float
        f1: float
        mean_iou: float            # over matched pairs (0.0 if none)
        mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
        mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

        def as_dict(self) -> dict:
            return self.__dict__.copy()


    def score(preds: List[Interval], gts: List[Interval],
              iou_threshold: float = 0.5,
              match_result: Optional[MatchResult] = None) -> Score:
        """Compute precision/recall/F1, mean matched IoU, and boundary errors.

        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
        extracting the failure lists); otherwise matching is done here.
        """
        mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

        tp = len(mr.matches)
        fp = len(mr.unmatched_pred_indices)
        fn = len(mr.unmatched_gt_indices)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0

        if mr.matches:
            mean_iou = sum(m.iou for m in mr.matches) / tp
            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
        else:
            mean_iou = mean_start_err = mean_end_err = 0.0

        return Score(
            iou_threshold=iou_threshold,
            num_pred=len(preds), num_gt=len(gts),
            true_positives=tp, false_positives=fp, false_negatives=fn,
            precision=precision, recall=recall, f1=f1,
            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
```

```
        )


    def pr_curve(preds: List[Interval], gts: List[Interval],
                 thresholds: Optional[List[float]] = None) -> List[Score]:
        """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).

        Lets you see strict-vs-lenient behaviour rather than a single operating point.
        """
        if thresholds is None:
            thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
        return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

10. assistant (2026-06-14T08:59:29.775Z)

Now I have a clear understanding of the APIs. Let me examine the `fusion_compare.py` module to understand how `compare` is invoked with learned variants and how it returns results, to mirror the pattern correctly.

11. user (2026-06-14T08:59:30.483Z)

```
1       """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3       The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4       and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5       no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6       designed seam): the person features are extra columns the learned LogisticRegression
weights,
7       so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9       Run (no GPU):
10          python -m backend.eval.fusion_compare --features-dir output --record
11      """
12      from __future__ import annotations
13
14      import argparse
15      import glob
16      import json
17      import os
18      from typing import Any, Dict, List
19
20      from backend.eval import experiment
21      from backend.eval.eval_stats import paired_delta_ci
22      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25      def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26          """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27          (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28          by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29          by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30          by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31          vids = [p["video"] for p in res["variant_a"]["per_video"]]
32          return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35      def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
```

```
        """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
        feature columns + golden gts). The harness derives labels from gts itself."""
        videos: List[experiment.VideoCandidates] = []
        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_FEATURE_NAMES:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_FEATURE_NAMES:
                features[fn] = [float(c["person"][fn]) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                     features=features, gts=gts))
        return videos


    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
                threshold: float = 0.5) -> Dict[str, Any]:
        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
                                          threshold=threshold)
        fusion = experiment.Variant(name="shuttle_person_fusion",
                                    feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
                                    threshold=threshold)
        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)


    def format_result(res: Dict[str, Any], iou: float) -> str:
        a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
        ds = delta_stats(res)
        st = ds["sign_test"]
        lines = [
            f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
            f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
            f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
            f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
            f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
            f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
            f"| CI clears 0: {ds['ci_excludes_zero']}",
            "  per-video ?F1:",
        ]
        for pv in res["per_video_delta"]:
            lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
        return "\n".join(lines)


    def main() -> None:
        ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
        ap.add_argument("--features-dir", default="output")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
        args = ap.parse_args()

        videos = load_videos(args.features_dir)
```

```
92          if len(videos) < 2:
93              raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94          res = compare(videos, iou=args.iou)
95          print(format_result(res, args.iou))
96          if args.record:
97              row = experiment.record_experiment(res)
98              print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})")
99
100
101     if __name__ == "__main__":
102         main()
103
```

12. assistant (2026-06-14T08:59:39.812Z)

Now I fully understand the harness. Key design points for `learning_curve`:

- `compare()` runs honest LOVO on a corpus, but the task asks for a **held-out split** semantic: fit/score on k videos, evaluate on the remaining held-out videos. This is different from LOVO (which is leave-one-out within a single corpus).
- The approach: at each k, take the first k videos as the "personalization corpus" and the remaining `len(videos)-k` as held-out. Run `compare(control, candidate)` on the held-out set, but learned variants need to be **fit on the k corpus videos** and **scored on held-out**.

Let me think carefully about how to use the existing primitives. The `predict(variant, vc, model=...)` lets me supply a fitted model. `experiment._train(variant, train_videos, iou)` fits a learned variant on a set. So the held-out split for a learned candidate is: train on the k corpus videos, predict each held-out video with that model, score against gts. For a static control variant, just predict each held-out video directly (no training). This reuses the harness primitives cleanly.

Let me verify the environment (ruff/mypy/pytest availability) and check git worktree state.

13. user (2026-06-14T08:59:42.067Z)

```
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
pytest 9.0.3
C:/Users/avidu/Projects/badminton-highlight-indexer  77082f4 [master]
C:/Users/avidu/Projects/bhi-puser                    8680dfe [feat/per-user-promotion-gate]
```

14. assistant (2026-06-14T08:59:45.806Z)

There's already a sibling worktree at `bhi-puser` on a different branch (`feat/per-user-promotion-gate`) ? a sibling session. My target is `bhi-pueval` with branch `feat/per-user-eval`, which is distinct. Let me check the master HEAD and whether my branch/worktree path already exist.

15. user (2026-06-14T08:59:47.132Z)

```
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)
---branches---
---path---
absent
```

16. assistant (2026-06-14T08:59:49.985Z)

`origin/master` is at `77082f4` (#140 ? newer than my snapshot). Branch `feat/per-user-eval` doesn't exist and the target path is absent. Let me create the isolated worktree off `origin/master`.

17. user (2026-06-14T08:59:51.449Z)

```
Preparing worktree (new branch 'feat/per-user-eval')
branch 'feat/per-user-eval' set up to track 'origin/master'.
HEAD is now at 77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor
(#140)
```

18. assistant (2026-06-14T08:59:56.099Z)

Worktree created at `bhi-pueval` on `feat/per-user-eval` off `origin/master` (77082f4). Let me confirm the eval files are identical in the new worktree (origin/master may have evolved past my read), then check existing eval module patterns for `__future__`/typing conventions.

19. user (2026-06-14T08:59:57.829Z)

```
feat/per-user-eval
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)
---diff experiment.py---
(empty diff above = identical to what I read)
```

20. assistant (2026-06-14T09:00:08.157Z)

Files are identical. Now let me write the implementation. I'll design `learning_curve` carefully:

Design:
- Input: a single user's `videos: Sequence[VideoCandidates]`, a `control_variant` and `candidate_variant`.
- Sweep `k = min_k .. len(videos)-1`. At each k, fit/score on the first `k` videos, evaluate on the remaining `len(videos)-k` held-out videos.
- For each held-out video, compute control F1 and candidate F1:
  - **Static variant** (no learned filter): `predict(variant, vc)` directly ? no training needed.
  - **Learned variant** (`feature_names` set, no pinned model): train on the k corpus videos via `experiment._train`, then predict each held-out video with that fitted model.
  - **Pinned model** (`classifier_model` set): score with the shipped model directly.
- Per-k record: pair control vs candidate F1 by held-out video, run `paired_delta_ci` ? `mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test`. Plus `control_f1`/`candidate_f1` as the mean held-out F1.
- Headline `smallest_k_with_lift` = smallest k where `candidate_f1 > control_f1` ... actually the criterion is: CI clears 0 AND sign test agrees (B wins on more videos than it loses, with the same direction). Let me make "sign test agreeing" mean `wins > losses` (B beating A on a majority of held-out videos). The CI clearing 0 already enforces direction via `ci_excludes_zero` combined with `mean_delta > 0`.

Let me make the lift criterion precise: candidate beats control means `mean_delta > 0` AND `ci_excludes_zero` (CI above 0) AND sign test agrees (`wins > losses`). This is the honest, conservative criterion.

Now writing the module.

21. assistant (2026-06-14T09:00:42.078Z)

```
"""Held-out-user LEARNING-CURVE eval ? "<K videos to superior quality" as a release gate.

The personalization feature's promise is operational, not aspirational: *after the user
contributes K of their own videos, the personalized (candidate) recipe should beat the
generic (control) one on the user's STILL-UNSEEN footage.* This module turns that promise
into a falsifiable criterion by sweeping the corpus size ``k`` and asking, at each ``k``,
**does the candidate beat control on a held-out split, honestly?**

It is the per-user analogue of `fusion_compare`: pure-CPU, no new scoring/stats math. Every
fold reuses the shipped experiment primitives ?

  - `experiment.predict`  ? variant ? held-out intervals (static variants need no fit);
  - `experiment._train`   ? fit a learned variant on the corpus split (the same honest
```

```
    train-on-others step `evaluate_variant`'s LOVO uses, here train-on-the-k-corpus);
  - `metrics.score`       ? per-video held-out F1;
  - `eval_stats.paired_delta_ci` ? the honest paired ?F1 (bootstrap CI + exact sign test, C1).

**Split semantics (held-out USER split, NOT leave-one-out).** At each ``k`` the first ``k``
videos are the *personalization corpus* (fit/operating-point source) and the remaining
``len(videos) - k`` are *held-out* (scored, never seen during fit). This mirrors the product
reality ? the user has uploaded ``k`` videos and we want quality on their next one ? and keeps
``k`` and the evaluation set cleanly disjoint, so a measured lift can't be in-sample.

The headline ``smallest_k_with_lift`` is the smallest ``k`` at which the candidate beats
control with the **CI clearing 0 AND the sign test agreeing** (B wins on more held-out videos
than it loses) ? the same promotion-grade bar `compare()`'s honest block uses, never a bare
mean. ``None`` means the criterion was never met on this corpus (the honest negative).

Pure, offline, deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
`VideoCandidates`). See `docs/EXPERIMENT_HARNESS.md` + `docs/ANALYZERS_AND_RECONCILERS.md`
§13/C1.
"""
from __future__ import annotations

from typing import Any, Dict, List, Optional, Sequence

from backend.eval import eval_stats as _stats
from backend.eval import experiment
from backend.eval.classifier import LogisticRegression
from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
from backend.eval.metrics import score


def _fit_model(variant: Variant, corpus: Sequence[VideoCandidates],
               iou: float) -> Optional[LogisticRegression]:
    """The fitted model a variant needs to predict held-out videos, or None if it needs none.

    - **static variant** (no learned filter): None ? `predict` scores directly, no leakage.
    - **pinned model** (``classifier_model`` set): the shipped model, loaded as-is (no per-fold
      training ? reports "how the shipped artifact does on this user's held-out footage").
    - **learned recipe** (``feature_names``, no pinned model): fit on the ``corpus`` split via
      the harness's own honest training step. The corpus and held-out sets are disjoint, so the
      held-out scores are genuinely out-of-sample.
    """
    if variant.classifier_model is not None:
        return LogisticRegression.load(variant.classifier_model)
    if variant.feature_names:
        return experiment._train(variant, corpus, iou)
    return None


def _heldout_f1s(variant: Variant, corpus: Sequence[VideoCandidates],
                 heldout: Sequence[VideoCandidates], iou: float) -> List[float]:
    """Per-held-out-video F1 for ``variant``, fit on ``corpus`` (if learned), scored on
    ``heldout``. Held-out videos are video-aligned with the returned list (caller pairs B?A)."""
    model = _fit_model(variant, corpus, iou)
    f1s: List[float] = []
    for vc in heldout:
        assert vc.gts is not None  # guarded in learning_curve before any scoring
        preds = experiment.predict(variant, vc, model=model)
        f1s.append(score(preds, vc.gts, iou).f1)
    return f1s


def learning_curve(videos: Sequence[VideoCandidates],
                   control_variant: Variant,
                   candidate_variant: Variant,
                   *,
```

```
                iou: float = 0.5,
                min_k: int = 2,
                seed: int = 0) -> Dict[str, Any]:
    """Sweep corpus size ``k`` and report where the candidate first honestly beats control.

    For one user's ``videos``, for each ``k`` in ``min_k .. len(videos) - 1``: fit/score both
    variants on the first ``k`` videos (the personalization corpus) and evaluate on the
remaining
    held-out videos. Per-held-out-video F1s are paired B?A and summarised with
    ``eval_stats.paired_delta_ci`` (mean ?F1 + bootstrap CI + exact sign test).

    Returns ``{n_videos, min_k, iou, per_k, smallest_k_with_lift}`` where each ``per_k`` record
is::

        {k, n_heldout, control_f1, candidate_f1,
         mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}

    ``control_f1`` / ``candidate_f1`` are the mean held-out F1 of each variant at that ``k``.
    ``smallest_k_with_lift`` is the smallest ``k`` whose record shows a real lift ? candidate
    above control, the ?F1 CI clearing 0, AND the sign test agreeing (B wins on strictly more
    held-out videos than it loses) ? else ``None`` (the honest "never wins" answer).

    Pure + deterministic given ``seed`` (the only randomness is the CI bootstrap). Raises
    `ExperimentError` on unlabeled videos or a too-small corpus (no valid ``k`` to sweep).
    """
    n = len(videos)
    for vc in videos:
        if vc.gts is None:
            raise ExperimentError(
                f"{vc.name} has no golden labels; learning_curve needs labeled held-out videos")
    if min_k < 1:
        raise ExperimentError(f"min_k must be >= 1 (got {min_k})")
    if n <= min_k:
        # Every k in [min_k, n-1] needs k >= min_k corpus videos AND >= 1 held-out video, so
        # n must exceed min_k. Too-few videos ? no sweepable k; report an empty, honest curve.
        return {
            "n_videos": n,
            "min_k": min_k,
            "iou": iou,
            "per_k": [],
            "smallest_k_with_lift": None,
        }

    per_k: List[Dict[str, Any]] = []
    smallest_k_with_lift: Optional[int] = None
    for k in range(min_k, n):
        corpus = videos[:k]
        heldout = videos[k:]
        control_f1s = _heldout_f1s(control_variant, corpus, heldout, iou)
        candidate_f1s = _heldout_f1s(candidate_variant, corpus, heldout, iou)

        paired = _stats.paired_delta_ci(control_f1s, candidate_f1s, seed=seed)
        sign_test = paired["sign_test"]
        # Honest lift = the candidate is above control AND the CI clears 0 AND the sign test
        # agrees (B wins on strictly more held-out videos than it loses) ? the promotion bar.
        control_mean = sum(control_f1s) / len(control_f1s) if control_f1s else 0.0
        candidate_mean = sum(candidate_f1s) / len(candidate_f1s) if candidate_f1s else 0.0
        has_lift = (
            candidate_mean > control_mean
            and bool(paired["ci_excludes_zero"])
            and paired["mean_delta"] > 0.0
            and sign_test["wins"] > sign_test["losses"]
        )
        per_k.append({
            "k": k,
```

```
            "n_heldout": len(heldout),
            "control_f1": control_mean,
            "candidate_f1": candidate_mean,
            "mean_delta": paired["mean_delta"],
            "ci_low": paired["ci_low"],
            "ci_high": paired["ci_high"],
            "ci_excludes_zero": bool(paired["ci_excludes_zero"]),
            "sign_test": sign_test,
        })
        if has_lift and smallest_k_with_lift is None:
            smallest_k_with_lift = k

    return {
        "n_videos": n,
        "min_k": min_k,
        "iou": iou,
        "per_k": per_k,
        "smallest_k_with_lift": smallest_k_with_lift,
    }
```

22. user (2026-06-14T09:00:42.099Z)

```
File created successfully at: C:/Users/avidu/Projects/bhi-pueval/backend/eval/per_user_eval.py
(file state is current in your context ? no need to Read it back)
```

23. assistant (2026-06-14T09:00:47.893Z)

Now let me check the `LogisticRegression` and `classifier` module's public surface to confirm `_train`/`predict` work as I expect with synthetic features, and confirm `score(...).f1`. Let me look at the classifier to understand feature behaviour for crafting tests.

24. user (2026-06-14T09:00:48.176Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
20
21      class LogisticRegression:
22          def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23              self.lr = lr
24              self.epochs = epochs
25              self.l2 = l2
26              self.w: Optional[np.ndarray] = None
27              self.b: float = 0.0
28              self.mean: Optional[np.ndarray] = None
29              self.std: Optional[np.ndarray] = None
30              self.feature_names: Optional[List[str]] = None
31
```

```
        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

```

```
97          @classmethod
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101             m.b = float(d.get("b", 0.0))
102             m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103             m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104             m.feature_names = d.get("feature_names")
105             return m
106
107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115
```

25. assistant (2026-06-14T09:00:51.788Z)

The classifier uses numpy. Note: the dev container "lacks GPU / torch / cv2 / numpy" per CLAUDE.md ? but ruff/mypy/pytest are installed. Let me check whether numpy is actually available, since the learned-variant path (and the fusion test) needs it.

26. user (2026-06-14T09:00:56.971Z)

numpy 2.4.6

27. assistant (2026-06-14T09:01:08.404Z)

numpy is available (cv2/torch are not, which is fine ? the eval path is pure-CPU and only needs numpy). This means I can build tests with the learned-variant path exactly like `test_fusion_compare.py`.

Now let me design the tests. I'll mirror the synthetic-VideoCandidates pattern. Strategy:

- **Separating-feature corpus**: a `players_in_court`-style feature that cleanly separates positive (rally) from negative (junk) candidates. The candidate variant has `feature_names=["sep"]`; the control variant is static (no filter, no gate) ? keeps everything ? over-fires ? low precision/F1. The candidate, once trained on enough corpus videos, filters junk ? high F1. So above some k the candidate wins. This mirrors `test_fusion_compare`.

But wait ? I need control to be *worse* and candidate *better*. A learned candidate needs k>=1 corpus video to train. Each held-out video must have both pos and neg candidates so control's F1 is mediocre (over-firing) and candidate's is high.

Let me think about what makes the curve interesting (lift appears only above some k). With logistic regression and `balanced` class weighting, even 1 corpus video may train a decent separator if the signal is clean. To make lift emerge only at higher k, I can use a *noisy* signal where a single video's training is unreliable but pooling more videos cleans it up. However that introduces fragility.

Simpler, robust approach for test (1): make the corpus videos progressively more informative isn't needed ? instead make held-out F1 for candidate reliably high and control reliably lower, so `smallest_k_with_lift == min_k` (found + sensible: a small k). The task says "candidate clearly wins above some k ? smallest_k_with_lift is found + sensible". `min_k` itself is a sensible found value. Let me verify this works empirically rather than over-engineering.

Let me first write a quick probe to see how control vs candidate behave on synthetic data.

28. user (2026-06-14T09:01:17.433Z)

```
smallest_k_with_lift= 2
  k=2 nheld=4 ctrl=0.667 cand=1.000 dF1=+0.333 CI=[+0.333,+0.333] clears0=True W/L=4/0
  k=3 nheld=3 ctrl=0.667 cand=1.000 dF1=+0.333 CI=[+0.333,+0.333] clears0=True W/L=3/0
  k=4 nheld=2 ctrl=0.667 cand=1.000 dF1=+0.333 CI=[+0.333,+0.333] clears0=True W/L=2/0
  k=5 nheld=1 ctrl=0.667 cand=1.000 dF1=+0.333 CI=[+0.333,+0.333] clears0=True W/L=1/0
```

29. assistant (2026-06-14T09:01:24.842Z)

The clean-separation corpus yields `smallest_k_with_lift == 2` (= min_k), candidate clearly wins (F1 1.000 vs control 0.667), CI clears 0, sign test all-wins. Note at k=5 there's only 1 held-out video ? CI is degenerate point interval but `ci_excludes_zero` is True since delta=0.333>0.

This satisfies test (1). Now for a more compelling "lift appears only above some k" demonstration, let me probe a noisy corpus where a single training video is insufficient but pooling helps. Let me test the "never wins" case (2) and a noisy case.

30. user (2026-06-14T09:01:33.891Z)

```
NEVER-WINS smallest_k_with_lift= None
  k=2 ctrl=0.667 cand=0.667 dF1=+0.000 clears0=False
  k=3 ctrl=0.667 cand=0.667 dF1=+0.000 clears0=False
  k=4 ctrl=0.667 cand=0.667 dF1=+0.000 clears0=False
keys: ['candidate_f1', 'ci_excludes_zero', 'ci_high', 'ci_low', 'control_f1', 'k', 'mean_delta',
'n_heldout', 'sign_test']
sign_test keys: ['losses', 'n_effective', 'p_value', 'ties', 'wins']
EDGE n=2,min_k=2 -> None per_k len 0
EDGE n=1,min_k=2 -> None per_k len 0
EDGE n=0 -> None per_k len 0
```

31. assistant (2026-06-14T09:01:41.828Z)

All three behaviors confirmed:
- **Never-wins** (constant feature) ? `smallest_k_with_lift == None`, every k has delta 0 / CI not clearing 0.
- **Per-k record keys** are exactly the expected set including the honest CI/sign-test fields.
- **Edge** (n <= min_k, including n=0/1/2) ? empty `per_k`, `None` headline, no crash.

Now I want test (1) to also demonstrate "lift appears only above some k" more meaningfully (smallest_k > min_k), to exercise the sweep finding a non-trivial k. Let me construct a corpus where a single training video produces an unreliable separator but pooling cleans it. I'll use a feature that separates with per-video sign-flips so 1-2 videos mistrain but more videos converge. Let me probe.

32. user (2026-06-14T09:01:56.555Z)

```
MIXED smallest_k_with_lift= 3
  k=2 nheld=5 ctrl=0.667 cand=0.667 dF1=+0.000 clears0=False W/L=0/0
  k=3 nheld=4 ctrl=0.667 cand=1.000 dF1=+0.333 clears0=True W/L=4/0
  k=4 nheld=3 ctrl=0.667 cand=1.000 dF1=+0.333 clears0=True W/L=3/0
  k=5 nheld=2 ctrl=0.667 cand=1.000 dF1=+0.333 clears0=True W/L=2/0
  k=6 nheld=1 ctrl=0.667 cand=1.000 dF1=+0.333 clears0=True W/L=1/0
```

33. assistant (2026-06-14T09:02:03.617Z)

This is a compelling test (1): with the first corpus video carrying an inverted signal, at `k=2` the model is mistrained (no lift, delta 0), but at `k=3` the true-signal videos outvote it and the candidate cleanly wins (F1 1.000 vs 0.667, CI clears 0, all-wins sign test). So `smallest_k_with_lift == 3` ? found, sensible, and `> min_k`, demonstrating the sweep actually does its job. This is deterministic.

Now let me write the test file mirroring `test_fusion_compare.py`'s style.

34. assistant (2026-06-14T09:02:29.220Z)

```
"""GPU-free tests for the held-out-user learning-curve eval (backend.eval.per_user_eval).

Mirrors the synthetic-VideoCandidates pattern in test_fusion_compare.py: a learned candidate
variant weights a single feature column, a static control variant keeps everything. The
held-out-split sweep then answers "after how many of a user's videos does the personalized
recipe honestly beat the generic one?" ? found only when the ?F1 CI clears 0 AND the sign test
agrees. Pure-CPU (numpy only), deterministic.
"""
import pytest

from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
from backend.eval.per_user_eval import learning_curve

SEP = "players_in_court"  # the one separating feature the learned candidate weights
```

Per-k record keys the honest curve must carry (incl. the C1 CI + sign-test fields).

```
_EXPECTED_KEYS = {
    "k", "n_heldout", "control_f1", "candidate_f1",
    "mean_delta", "ci_low", "ci_high", "ci_excludes_zero", "sign_test",
}
_SIGN_TEST_KEYS = {"wins", "losses", "ties", "n_effective", "p_value"}


def _video(name: str, *, separating: bool = True) -> VideoCandidates:
    """A 4-candidate video: 2 positives (match the gts) + 2 negatives (junk).

    ``separating`` True  ? the feature is high on rallies / low on junk (true signal);
    ``separating`` False ? the feature is INVERTED (high on junk), which mistrains a model
    fit on too few videos. Held-out videos are always true-signal so they can be scored.
    """
    intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
    sep = [4.0, 4.0, 0.0, 0.0] if separating else [0.0, 0.0, 4.0, 4.0]
    gts = [(0.0, 10.0), (20.0, 30.0)]
    return VideoCandidates(name=name, intervals=intervals, features={SEP: sep}, gts=gts)


def _control() -> Variant:
    """Static control: no learned filter, no gate ? keeps every candidate (over-fires)."""
    return Variant(name="generic")


def _candidate() -> Variant:
    """Learned candidate: weights the separating feature, fit per corpus split (LOVO-style)."""
    return Variant(name="personalized", feature_names=[SEP])


def test_candidate_wins_above_some_k():
    """A corpus where the candidate clearly wins above some k ? smallest_k_with_lift found.

    The first corpus video carries an INVERTED signal, so a model fit at k=2 misfilters (no
    lift); once the true-signal videos outvote it (k>=3) the candidate cleanly beats control.
    """
    videos = [_video("v0", separating=False)] + [_video(f"v{i}") for i in range(1, 7)]
    res = learning_curve(videos, _control(), _candidate(), iou=0.5, min_k=2, seed=0)

    k = res["smallest_k_with_lift"]
    assert k is not None
    assert res["min_k"] <= k < res["n_videos"]          # a sensible, in-range corpus size
    assert k == 3                                        # deterministic: lift emerges at k=3

    # At the winning k the candidate is strictly above control with the honest bar met.
    rec = next(r for r in res["per_k"] if r["k"] == k)
    assert rec["candidate_f1"] > rec["control_f1"]
    assert rec["mean_delta"] > 0.0
```

```
    assert rec["ci_excludes_zero"] is True
    assert rec["sign_test"]["wins"] > rec["sign_test"]["losses"]


def test_candidate_never_wins_returns_none():
    """A corpus where the feature carries NO signal (constant) ? smallest_k_with_lift is
None."""
    videos = [
        VideoCandidates(name=f"v{i}",
                        intervals=[(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)],
                        features={SEP: [1.0, 1.0, 1.0, 1.0]},          # constant ? non-
separating
                        gts=[(0.0, 10.0), (20.0, 30.0)])
        for i in range(6)
    ]
    res = learning_curve(videos, _control(), _candidate(), iou=0.5, min_k=2, seed=0)

    assert res["smallest_k_with_lift"] is None
    # Every k records a non-winning fold: no delta and a CI that does not clear 0.
    for rec in res["per_k"]:
        assert rec["mean_delta"] == pytest.approx(0.0)
        assert rec["ci_excludes_zero"] is False


def test_per_k_records_carry_honest_fields():
    """Each per-k record carries the expected keys + a well-formed CI and sign test."""
    videos = [_video(f"v{i}") for i in range(6)]
    res = learning_curve(videos, _control(), _candidate(), iou=0.5, min_k=2, seed=0)

    assert res["per_k"]                                  # min_k=2, n=6 ? ks 2..5
    assert [r["k"] for r in res["per_k"]] == [2, 3, 4, 5]
    for rec in res["per_k"]:
        assert set(rec) == _EXPECTED_KEYS
        assert set(rec["sign_test"]) == _SIGN_TEST_KEYS
        assert rec["n_heldout"] == res["n_videos"] - rec["k"]
        # CI brackets the mean delta (±fp slack), as eval_stats.paired_delta_ci guarantees.
        assert rec["ci_low"] - 1e-9 <= rec["mean_delta"] <= rec["ci_high"] + 1e-9
        assert isinstance(rec["ci_excludes_zero"], bool)


def test_too_few_videos_handled_gracefully():
    """n <= min_k ? no sweepable k: an empty curve + None headline, never a crash."""
    one = [_video("solo")]
    two = [_video("a"), _video("b")]
    for videos in (one, two):
        res = learning_curve(videos, _control(), _candidate(), min_k=2, seed=0)
        assert res["per_k"] == []
        assert res["smallest_k_with_lift"] is None
        assert res["n_videos"] == len(videos)
    # zero videos is likewise graceful
    assert learning_curve([], _control(), _candidate(), min_k=2)["per_k"] == []


def test_unlabeled_video_raises():
    """A video with no golden labels can't be a held-out target ? ExperimentError."""
    videos = [_video("v0"), VideoCandidates(name="unlabeled", intervals=[(0.0, 1.0)],
                                            features={SEP: [1.0]}, gts=None)]
    with pytest.raises(ExperimentError):
        learning_curve(videos, _control(), _candidate(), min_k=2)
```

35. user (2026-06-14T09:02:29.242Z)

```
File created successfully at: C:/Users/avidu/Projects/bhi-pueval/tests/test_per_user_eval.py
(file state is current in your context ? no need to Read it back)
```

36. assistant (2026-06-14T09:02:31.974Z)

Now let me run the full verification suite: ruff, mypy, and pytest.

37. user (2026-06-14T09:02:51.117Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
=== PYTEST ===
....................                                                     [100%]
20 passed in 3.59s
```

38. assistant (2026-06-14T09:02:57.921Z)

All green:
- **ruff**: All checks passed
- **mypy**: Success: no issues found in 1 source file (the 3 `note:` lines are advisory annotation-unchecked notes about *other* files, not errors)
- **pytest**: 20 passed (5 new per_user_eval tests + 8 fusion_compare + 7 eval_stats)

Let me confirm I imported `experiment._train` (a private helper) cleanly ? mypy passed, but let me double-check there's no ruff concern with the private import, and run the broader test suite touching the eval package to ensure no regression. Let me also verify the full `run_all_tests.py` isn't needed but at least the eval tests pass broadly.

39. user (2026-06-14T09:03:12.728Z)

```
........................................................................ [ 48%]
........................................................................ [ 96%]
.....                                                                    [100%]
149 passed, 412 deselected in 12.17s
```

40. assistant (2026-06-14T09:03:16.126Z)

149 related tests pass, no regressions. Now let me verify the branch state and stage only my explicit files before committing.

41. user (2026-06-14T09:03:18.021Z)

```
feat/per-user-eval
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)
---only my commits should appear vs origin/master---
(empty = clean base)
---status---
?? backend/eval/per_user_eval.py
?? tests/test_per_user_eval.py
```

42. assistant (2026-06-14T09:03:21.001Z)

Clean base (HEAD == origin/master, no sibling commits), only my two new files untracked. Let me stage the explicit paths and commit.

43. user (2026-06-14T09:03:21.999Z)

```
warning: in the working copy of 'backend/eval/per_user_eval.py', LF will be replaced by CRLF the
next time Git touches it
```

```
warning: in the working copy of 'tests/test_per_user_eval.py', LF will be replaced by CRLF the
next time Git touches it
A  backend/eval/per_user_eval.py
A  tests/test_per_user_eval.py
```

44. assistant (2026-06-14T09:03:25.020Z)

```
Both files staged (the LF/CRLF warnings are benign ? Git's autocrlf normalization on Windows).
Now committing.
```

45. user (2026-06-14T09:03:33.997Z)

```
[feat/per-user-eval 6a43914] feat(eval): held-out-user learning-curve eval ? "<K videos to
superior quality" as a release gate
 2 files changed, 280 insertions(+)
 create mode 100644 backend/eval/per_user_eval.py
 create mode 100644 tests/test_per_user_eval.py
```

46. assistant (2026-06-14T09:03:39.123Z)

```
Committed as `6a43914`. Now push and open the PR.
```

47. user (2026-06-14T09:03:41.897Z)

```
remote:
remote: Create a pull request for 'feat/per-user-eval' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/per-user-eval
remote:
branch 'feat/per-user-eval' set up to track 'origin/feat/per-user-eval'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      feat/per-user-eval -> feat/per-user-eval
```

48. assistant (2026-06-14T09:03:44.262Z)

```
Pushed. Now creating the PR against master.
```

49. user (2026-06-14T09:04:01.839Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/142
```

50. assistant (2026-06-14T09:04:07.583Z)

```
PR created: **#142**. Let me do a final confirmation of the PR state (it targets master and
contains only my commit) and re-run the exact required verification triplet in the worktree one
more time to report exact results.
```

51. user (2026-06-14T09:04:17.566Z)

```
{"base":"master","files":["backend/eval/per_user_eval.py","tests/test_per_user_eval.py"],"head":
"feat/per-user-eval","number":142,"state":"OPEN","url":"https://github.com/avidullu/badminton-
highlight-indexer/pull/142"}
=== FINAL VERIFY ===
[ruff]
All checks passed!
[mypy]
Success: no issues found in 1 source file
[pytest]
....................                                                     [100%]
20 passed in 3.60s
```

52. assistant (2026-06-14T09:04:32.448Z)

```
Done. The PR is open, targets `master`, contains exactly the two intended files, and all checks
```

are green.

Summary

Implemented a held-out-user learning-curve eval harness that operationalizes "<K videos to superior quality" as a falsifiable release criterion for personalization.

PR: #142 ? https://github.com/avidullu/badminton-highlight-indexer/pull/142 (OPEN, base `master`, NOT merged)

Worktree: `C:/Users/avidu/Projects/bhi-pueval` (branch `feat/per-user-eval`, off `origin/master` @ `77082f4`; single commit `6a43914`)

Files changed (2, +280):
- `backend/eval/per_user_eval.py` ? `learning_curve(videos, control_variant, candidate_variant, *, iou=0.5, min_k=2, seed=0)`
- `tests/test_per_user_eval.py` ? 5 GPU-free synthetic-`VideoCandidates` tests

Design notes:
- **Held-out USER split (not LOVO):** at each `k`, first `k` videos = personalization corpus (fit/operating-point), remaining `len(videos)-k` = held-out (scored). Disjoint ? genuinely out-of-sample.
- Reuses harness primitives only ? `experiment.predict`, `experiment._train` (the honest train-on-corpus fit), `metrics.score`, and `eval_stats.paired_delta_ci` for the ?F1 CI + exact sign test. No new scoring/stats math.
- Per-k record: `{k, n_heldout, control_f1, candidate_f1, mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}`.
- Headline `smallest_k_with_lift` = smallest `k` where candidate beats control **with CI clearing 0 AND sign test agreeing** (wins > losses), else `None`. Pure-CPU, deterministic, no file I/O.
- Test (1) is non-trivial: an inverted first corpus video mistrains the model at `k=2`, true-signal videos outvote it at `k=3`, so lift deterministically emerges at `k=3` (exercises the sweep, not just `min_k`).

Exact verification results (all pass):
- `ruff check backend/eval/per_user_eval.py tests/test_per_user_eval.py` ? **All checks passed!**
- `mypy backend/eval/per_user_eval.py` ? **Success: no issues found in 1 source file**
- `pytest tests/test_per_user_eval.py tests/test_fusion_compare.py tests/test_eval_stats.py -q` ? **20 passed**
- (regression sanity: `pytest -k "eval or fusion or experiment"` ? 149 passed, 0 failures)

Branch-safety: branched explicitly from `origin/master` in an isolated worktree; re-verified `git log origin/master..HEAD` listed only my commit before committing; staged explicit paths only (no `git add -A`). One pre-existing sibling worktree (`bhi-puser` on `feat/per-user-promotion-gate`) was left untouched.

One observation worth flagging: I import `experiment._train` (a leading-underscore helper) from `per_user_eval.py`. It's the cleanest reuse of the harness's honest training step and `fusion_compare` already reaches into harness internals similarly, but if you'd prefer `_train` promoted to a public name that's a small follow-up.